Ensemble Model

PGR216: AI and Cyber Security Applications

Instructor: Mohib Ullah

23 March 2026

Agenda

- Ensemble Model
- Bagging
- Boosting
- Stacking
- Central Limit theorem

Why Not a Single Model?

Limitations of single models:

- Sensitive to noise and sampling variation
- May overfit (high variance) or underfit (high bias)
- Performance depends heavily on model choice and hyperparameters
- Different models make different mistakes

- **Key question:**
  Can we combine multiple models to get better performance?

Error Reduction View

- Revisit Bias-Variance of AI models

- When it comes to mitiagation strategies, we discussed the following:
  - Regularization
  - Model complexity
  - control
  - Cross-validation
  - More data
  - Feature engineering
  - **Ensembles**

Intuition Behind Ensemble Learning

Core idea:

A group of diverse weak models can form a strong model.

- Analogy:
  - Medical diagnosis by multiple doctors
  - Crowd wisdom
  - Jury decision making

- Benefits:
  - More stable predictions
  - Higher accuracy
  - Reduced risk of choosing a poor single model

Error Reduction View

Prediction error consists of:

- Bias (systematic error)
- Variance (sensitivity to data)
- Noise (irreducible)

- **Ensembles help by:**
  - Averaging out variance
  - Reducing bias (boosting)
  - Making predictions more robust to noise

What is Ensemble Learning?

An ensemble model trains multiple base learners and combine their prediction to reduce variance, enhance prediction and helps mitigate overfitting.

For classification:

- Majority vote or probability averaging

For regression:

- Average or weighted average

- Common base learners:
  - Linear models
  - SVMs
  - Neural networks

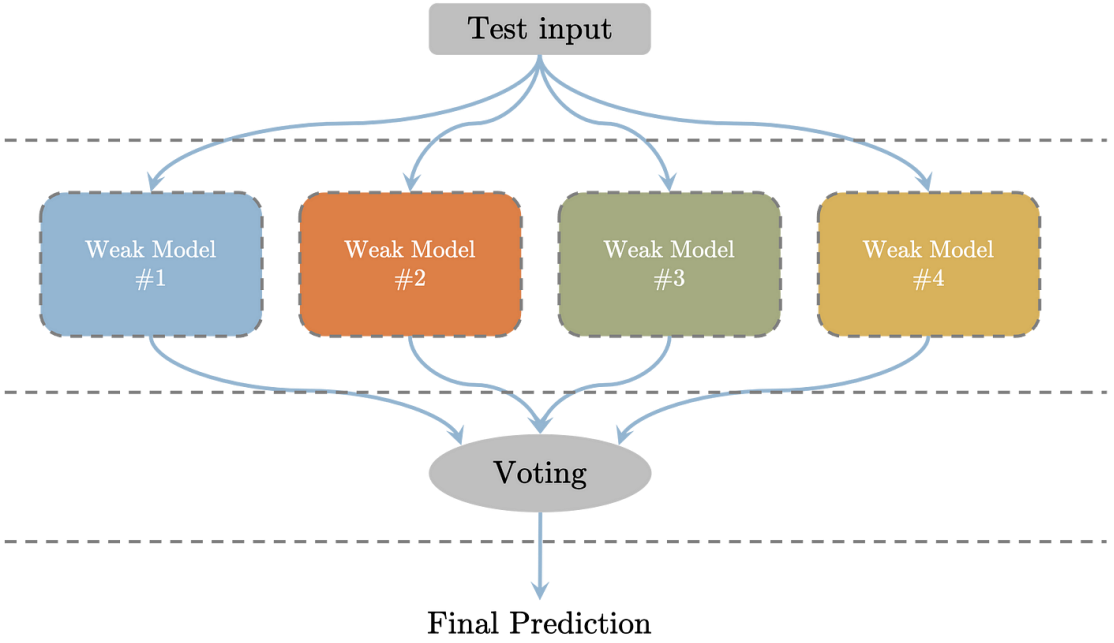
Why Use Ensemble Learning?

No single model is perfect! Combining models can:

- Reduce bias
- Reduce variance
- Improve generalization

Types of Ensemble Methods

- Bagging (Bootstrap Aggregating)
- Boosting (Sequential Learning)
- Stacking (Meta-Learning)



Bagging (Bootstrap Aggregating)

- Randomly sample data and train multiple models ()
- Average predictions for stability
- Example: Random Forest

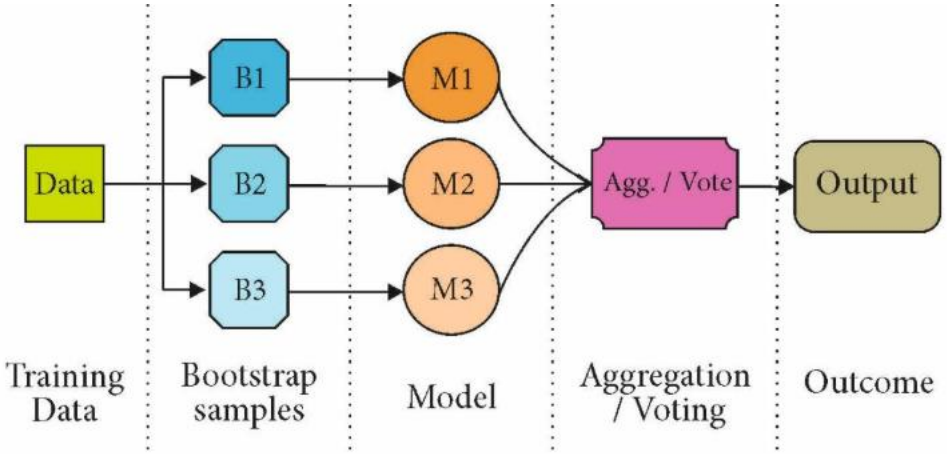


Figure courtesy: https://analyticsindiamag.com/deep-tech/guide-to-ensemble-methods-bagging-vs-boosting/

Bagging (Bootstrap Aggregating)

- Bootstrap Sampling (Random Resampling)
  - Take multiple random samples with replacement from the original dataset.
  - Each sample (bootstrap sample) has the same size as the original dataset but may contain duplicate data points.

- Train Multiple Models
  - Train a separate base model on each bootstrap sample.
  - These base models are often weak learners (e.g., decision trees).

- Aggregate Predictions
  - For regression: Average the predictions of all models.
  - For classification: Use majority voting (the most predicted class wins).

- Final Prediction
  - The combined model produces a more stable and accurate prediction.

Sampling With Replacement vs. Without Replacement

- Sampling is how we select data points from a dataset. There are two main types:

1. Sampling With Replacement (Used in Bagging)
   - Each selected data point can be chosen multiple times.
   - Example: If we pick 10 students from a class of 30, a student can appear more than once in our sample.
   - Common in bootstrap sampling because it allows diversity among different samples.

2. Sampling Without Replacement
   - Each selected data point can only be chosen once.
   - Example: If we pick 10 students from a class of 30, each student appears at most once in our sample.
   - Used in methods like cross-validation and feature selection to ensure unique data points.

Size of bootstrap

It’s not strictly necessary to have same size bootstrap as the original data, but it is commonly done for stability reasons. However:

Same Size as Original Dataset (Common in Bagging)

- Ensures each model learns from a complete dataset size.
- Some data points are repeated, but around 63% of unique data points are included on average.

Smaller Bootstrap Sample

- Can be used to reduce computational cost.
- Works well when the dataset is very large.

Larger Bootstrap Sample

- Less common but possible.
- May lead to overfitting since some points are oversampled.

Popular Model

- Random Forest
- Bagged decision trees
- Bagged SVM

Boosting (Sequential Learning)

- Train models sequentially, focusing on mistakes
- Weigh harder-to-classify samples more
- Examples: AdaBoost, XGBoost, Gradient Boosting

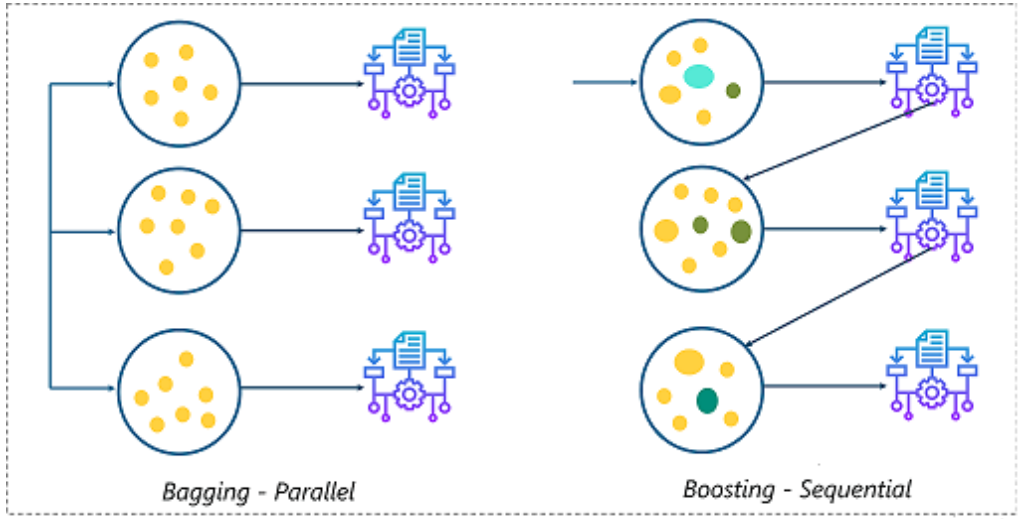


Figure courtesy: https://medium.com/@HalderNilimesh/boosting-your-machine-learning-models-a-comprehensive-guide-to-understanding-and-implementing-7a2737e9251b

Boosting (Sequential Learning)

- Initialize Equal Weights
  - Each training example starts with an equal weight.
- Train the First Weak Model
  - A simple model is trained on the weighted dataset.
- Identify Misclassified Data
  - The model predicts the output, and incorrect predictions are given higher weights, so the next model focuses more on them.
- Train the Next Model on the Updated Data
  - A new model is trained, prioritizing the previous model's mistakes.
- Repeat for Multiple Rounds
  - This process continues for several iterations, with each model improving upon the previous one.
- Final Prediction: Weighted Voting or Summation
  - In classification, models vote, with better models getting more influence.
  - In regression, predictions are combined based on weights.

Boosting Example

Scenario: Predicting Whether a Customer Will Buy a Product

- We have a dataset with customer features like age, income, and browsing behavior, and we want to predict whether they will buy a product (yes/no).

Step 1: Initialize Weights

- Each data point gets an equal weight (1/N where N is the number of data points).

Step 2: Train the First Weak Model

- A shallow decision tree (stump) is trained.
- It predicts well for some customers but misclassifies others.

Step 3: Increase Weights for Misclassified Customers

- The customers misclassified by the first model get higher weights.

Step 4: Train the Second Model with Updated Weights

- The second model focuses more on previously misclassified customers.

Step 5: Repeat the Process

- We continue training multiple models, each improving upon the last.

Step 6: Combine the Models' Predictions

- Each model gets a weight based on its accuracy.
- Final predictions are made using weighted voting (for classification) or weighted sum (for regression).

Bagging VS Boosting

- Focuses on Hard-to-Classify Examples.

- Unlike bagging, boosting prioritizes mistakes rather than averaging all predictions equally.

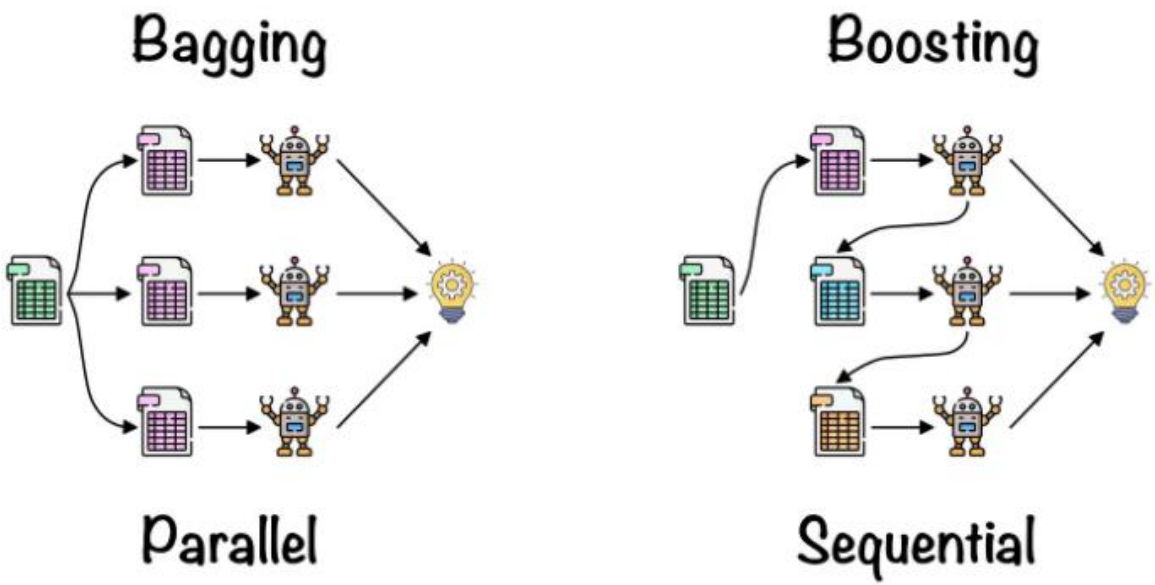


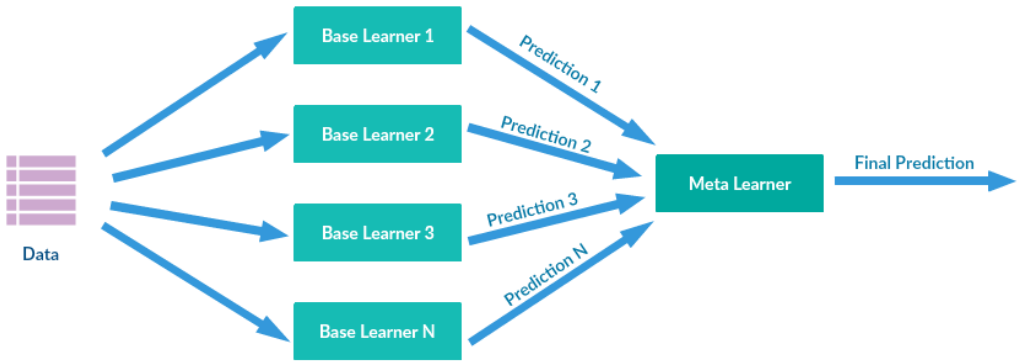
Figure Courtesy: https://www.appliedaicourse.com/blog/bagging-and-boosting-in-machine-learning/

Popular model

- AdaBoost
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost

Stacking (Meta-Learning)

- Combine different models into a higher-level model
- The meta-model learns how to best combine predictions
- More complex but powerful



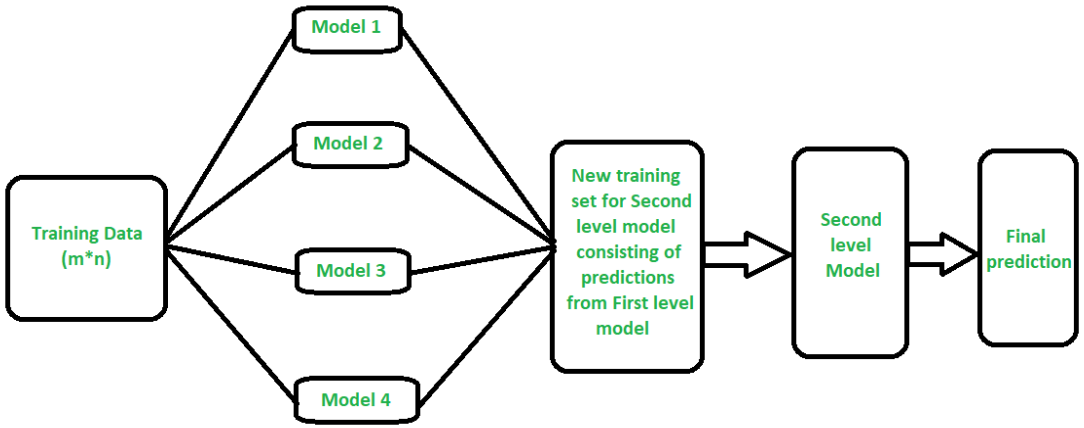


Figure courtesy: https://www.geeksforgeeks.org/stacking-in-machine-learning/

Stacking (Meta-Learning)

Train Multiple Base Models (First Layer)

- Choose different types of models (e.g., Decision Trees, SVM, linear model).
- Train them separately on the same dataset.
- Each model learns different patterns from the data.

Generate Predictions from Base Models

- Each base model predicts the output for training data.
- These predictions become a new dataset (meta-features).

Train a Meta-Model (Second Layer)

- The meta-model takes the predictions from base models as input.
- It learns how to **best combine** the outputs of base models.
- Common meta-models: **Logistic Regression, XGBoost.**

Final Prediction

- When making predictions, the base models give outputs.
- The meta-model combines them to make the final decision.

Stacking for Loan Approval Prediction

Scenario: Predict whether a loan application should be approved based on customer details.

Step 1: Train Base Models

- We train three different models:

Decision Tree → Good at capturing rules but may overfit.
Random Forest → Reduces overfitting with multiple trees.
Logistic Regression → Works well with linear relationships.

Step 2: Generate Predictions

- Each model gives a probability score (e.g., chance of loan approval).
- These predictions form a new dataset.

Customer	Decision Tree	Random Forest	Logistic Regression	Actual Approval
A	0.6	0.7	0.65	1
B	0.3	0.4	0.35	0
C	0.8	0.85	0.75	1

Stacking for Loan Approval Prediction

Step 3: Train a Meta-Model

- The meta-model (e.g., XGBoost) learns how to combine these predictions.
- It gives higher importance to the best base models.

Step 4: Final Prediction on New Data

- When a new loan application arrives, base models give predictions.
- The meta-model combines them to make the final decision.

Pros of Stacking

- **Combines Strengths of Different Models** – Decision trees capture non-linearity, logistic regression handles probabilities, and random forests reduce variance.

- **Reduces Overfitting** – Unlike bagging and boosting, stacking focuses on blending models effectively.

- **Improves Generalization** – The meta-model learns which base models work best in different scenarios.

Bias-Variance Tradeoff in Ensemble Learning

- Bagging reduces **variance** (overfitting)
- Boosting reduces **bias** (underfitting)
- Stacking leverages **both** for better generalization

Bias-Variance Decomposition (regression)

Assume:

$$y = f(x) + \varepsilon, \mathbb{E}[\varepsilon] = 0,\ \ Var(\varepsilon) = \sigma^2$$

- For a learned model $\hat{f}(x)$,the expected squared error at a point $x$is:

$$\mathbb{E}\left[\left(y - \hat{f}(x)\right)^2\right]$$

$$= \underbrace{\left(Bias\left[\hat{f}(x)\right]\right)^2}_{systematic\ error} + \underbrace{Var\left[\hat{f}(x)\right]}_{sensitivity\ to\ data} + \underbrace{\sigma^2}_{irreducible\ noise}$$

where:

$$\text{Bias}\left[\hat{f}(x)\right] = \mathbb{E}\left[\hat{f}(x)\right] - f(x)$$

$$\text{Var}\left[\hat{f}(x)\right] = \mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}\left[\hat{f}(x)\right]\right)^2\right]$$

Bagging: Why It Reduces Variance

- Consider an ensemble of M models:

$$\hat{f}_{ens}(x) = \frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x)$$

Variance of ensemble

Assume each model has:

- Variance = σ_f^2
- Pairwise correlation = ρ

Then:

$$\mathrm{Var}\left(\hat{f}_{ens}\right) = \frac{1}{M}\sigma_f^2 + \frac{M-1}{M}\rho\sigma_f^2$$

- Rewrite:

$$\mathrm{Var}\left(\hat{f}_{ens}\right) = \sigma_f^2\left(\rho + \frac{1-\rho}{M}\right)$$

Interpretation

- If models are **independent** ($\rho = 0$):

$$\text{Var} = \frac{\sigma_f^2}{M}$$

Huge reduction.

- If models are **perfectly correlated** ($\rho = 1$):

$$\text{Var} = \sigma_f^2$$

No improvement.

Stacking: Both Bias and Variance

Stacking trains:

$$\hat{f}_{ens}(x) = g\left(\hat{f}_1, (x), \ldots \hat{f}_M(x)\right)$$

The meta-model g learns optimal weights.

If diversity is high:

- Bias ↓ (models compensate each other)
- Variance ↓ (averaging effect)

But risk: overfitting if meta-model too complex.

Neural Networks as an Ensemble

- A neural network is itself an ensemble of neurons
- Many small units work together to learn patterns
- Dropout acts as an ensemble by training with different subsets of neurons

Dropout in Neural Networks

- Randomly removes neurons during training
- Forces the network to generalize better
- Similar to **“Bagging”** in ensemble learning

Central Limit Theorem (CLT)

- The Central Limit Theorem (CLT) states that if we take many random samples from a population and compute their averages, the distribution of these averages will approximate a normal distribution (bell curve), even if the original data is not normally distributed.

- **Key Idea:** When we combine multiple samples (or models), their errors cancel out, leading to a more stable and reliable prediction (reduce variance and stabilized predictions).

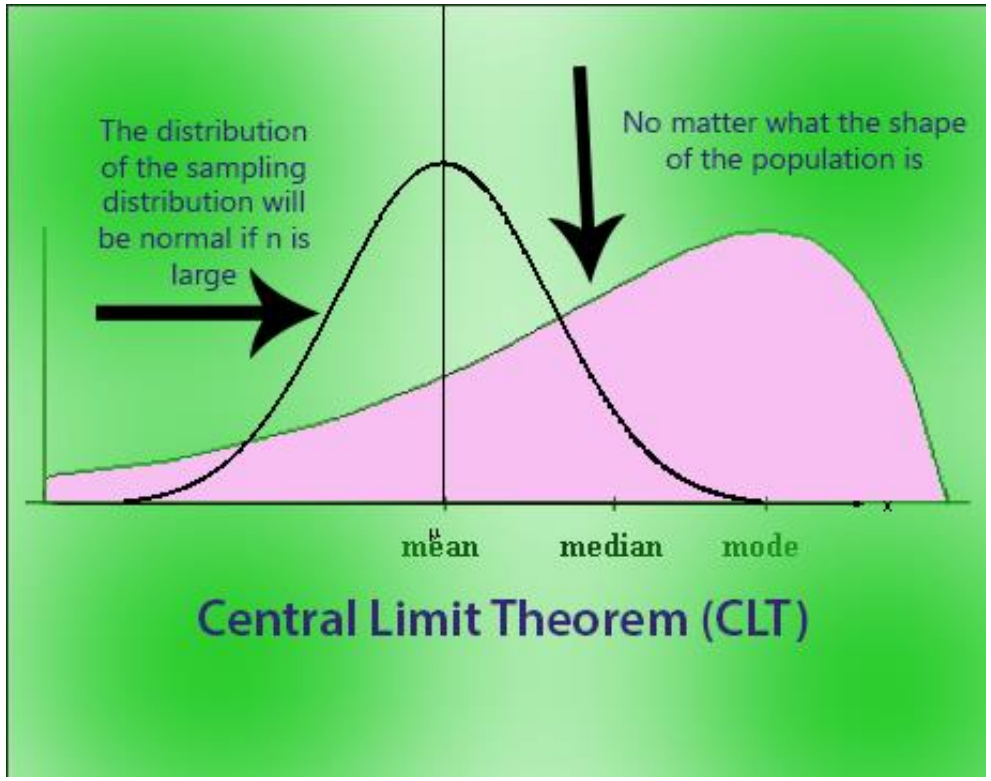


Figure courtesy: https://medium.com/@seema.singh/central-limit-theorem-simplified-46ddefeb13f3

CLT & Ensemble Learning

- When we combine multiple models, their errors average out
- Like CLT, ensemble learning stabilizes performance
- Helps achieve “better generalization”

Conclusion

- **Ensemble Learning Improves Accuracy** – By combining multiple models, we achieve better performance than individual models.
- **Reduces Variance & Bias** – Bagging lowers variance (Random Forest), boosting reduces bias (AdaBoost, XGBoost), and stacking balances both.
- **Inspired by Central Limit Theorem** – Aggregating predictions stabilizes outputs, just like averaging samples in CLT.
- **Neural networks** benefit from ensemble concept.
- **No One-Size-Fits-All Solution** – The best ensemble depends on the dataset and problem type.